

Sam	94	77	90
Katie	100	81	82
Bob	83	65	85

`T` returns a transposed *view* (not a copy) of the `DataFrame`.

Let's assume that rather than getting the summary statistics by student, you want to get them by test. Simply call `describe` on `grades.T`, as in:

[lick here to view code image](#)

```
In [30]: grades.T.describe()
Out[30]:
```

	Test1	Test2	Test3
count	5.00	5.00	5.00
mean	92.80	81.20	83.40
std	7.66	11.54	8.23
min	83.00	65.00	70.00
25%	87.00	77.00	82.00
50%	94.00	81.00	85.00
75%	100.00	87.00	90.00
max	100.00	96.00	90.00

To see the average of all the students' grades on each test, just call `mean` on the `T` attribute:

```
In [31]: grades.T.mean()
Out[31]:
```

Test1	92.8
Test2	81.2
Test3	83.4

dtype: float64

Sorting by Rows by Their Indices

You'll often sort data for easier readability. You can sort a `DataFrame` by its rows or columns, based on their indices or values. Let's sort the rows by their *indices* in *descending* order using `sort_index` and its keyword argument `ascending=False` (the default is to sort in *ascending* order). This returns a new `DataFrame` containing the sorted data:

[lick here to view code image](#)

```
In [32]: grades.sort_index(ascending=False)
```

```
Out[32]:
```

	Wally	Eva	Sam	Katie	Bob
Test3	70	90	90	82	85
Test2	96	87	77	81	65
Test1	87	100	94	100	83

Sorting by Column Indices

Now let's sort the columns into ascending order (left-to-right) by their column names. Passing the **axis=1 keyword argument** indicates that we wish to sort the *column* indices, rather than the row indices—`axis=0` (the default) sorts the *row* indices:

[lick here to view code image](#)

```
In [33]: grades.sort_index(axis=1)
Out[33]:
```

	Bob	Eva	Katie	Sam	Wally
Test1	83	100	100	94	87
Test2	65	87	81	77	96
Test3	85	90	82	90	70

Sorting by Column Values

Let's assume we want to see `Test1`'s grades in descending order so we can see the students' names in highest-to-lowest grade order. We can call the method **sort_values** as follows:

[lick here to view code image](#)

```
In [34]: grades.sort_values(by='Test1', axis=1, ascending=False)
Out[34]:
```

	Eva	Katie	Sam	Wally	Bob
Test1	100	100	94	87	83
Test2	87	81	77	96	65
Test3	90	82	90	70	85

The `by` and `axis` keyword arguments work together to determine which values will be sorted. In this case, we sort based on the column values (`axis=1`) for `Test1`.

Of course, it might be easier to read the grades and names if they were in a column, so we can sort the transposed `DataFrame` instead. Here, we did not need to specify the `axis` keyword argument, because `sort_values` sorts data in a specified column by default:

[lick here to view code image](#)

```
In [35]: grades.T.sort_values(by='Test1', ascending=False)
Out[35]:
```

	Test1	Test2	Test3
Eva	100	87	90
Katie	100	81	82
Sam	94	77	90
Wally	87	96	70
Bob	83	65	85

Finally, since you're sorting only `Test1`'s grades, you might not want to see the other tests at all. So, let's combine selection with sorting:

[lick here to view code image](#)

```
In [36]: grades.loc['Test1'].sort_values(ascending=False)
Out[36]:
```

Katie	100
Eva	100
Sam	94
Wally	87
Bob	83

Name: Test1, dtype: int64

Copy vs. In-Place Sorting

By default the `sort_index` and `sort_values` return a *copy* of the original `DataFrame`, which could require substantial memory in a big data application. You can sort the `DataFrame` *in place*, rather than *copying* the data. To do so, pass the keyword argument `inplace=True` to either `sort_index` or `sort_values`.

We've shown many pandas `Series` and `DataFrame` features. In the next chapter's Intro to Data Science section, we'll use `Series` and `DataFrames` for *data munging*—cleaning and preparing data for use in your database or analytics software.

7.15 WRAP-UP

This chapter explored the use of NumPy's high-performance `ndarrays` for storing and retrieving data, and for performing common data manipulations concisely and with reduced chance of errors with functional-style programming. We refer to `ndarrays` simply by their synonym, `arrays`.

he chapter examples demonstrated how to create, initialize and refer to individual elements of one- and two-dimensional arrays. We used attributes to determine an array's size, shape and element type. We showed functions that create arrays of 0s, 1s, specific values or ranges values. We compared list and array performance with the IPython `%timeit` magic and saw that arrays are up to two orders of magnitude faster.

We used array operators and NumPy universal functions to perform element-wise calculations on every element of arrays that have the same shape. You also saw that NumPy uses broadcasting to perform element-wise operations between arrays and scalar values, and between arrays of different shapes. We introduced various built-in array methods for performing calculations using all elements of an array, and we showed how to perform those calculations row-by-row or column-by-column. We demonstrated various array slicing and indexing capabilities that are more powerful than those provided by Python's built-in collections. We demonstrated various ways to reshape arrays. We discussed how to shallow copy and deep copy arrays and other Python objects.

In the Intro to Data Science section, we began our multisection introduction to the popular pandas library that you'll use in many of the data science case study chapters. You saw that many big data applications need more flexible collections than NumPy's arrays, collections that support mixed data types, custom indexing, missing data, data that's not structured consistently and data that needs to be manipulated into forms appropriate for the databases and data analysis packages you use.

We showed how to create and manipulate pandas array-like one-dimensional Series and two-dimensional DataFrames. We customized Series and DataFrame indices. You saw pandas' nicely formatted outputs and customized the precision of floating-point values. We showed various ways to access and select data in Series and DataFrames. We used method `describe` to calculate basic descriptive statistics for Series and DataFrames. We showed how to transpose DataFrame rows and columns via the `T` attribute. You saw several ways to sort DataFrames using their index values, their column names, the data in their rows and the data in their columns. You're now familiar with four powerful array-like collections—lists, arrays, Series and DataFrames—and the contexts in which to use them. We'll add a fifth—tensors—in the “Deep Learning” chapter.

In the next chapter, we take a deeper look at strings, string formatting and string methods. We also introduce regular expressions, which we'll use to match patterns in text. The capabilities you'll see will help you prepare for the “Natural Language

rocessing (NLP)” chapter and other key data science chapters. In the next chapter’s Intro to Data Science section, we’ll introduce pandas *data munging*—preparing data for use in your database or analytics software. In subsequent chapters, we’ll use pandas for basic time-series analysis and introduce pandas visualization capabilities.

. Strings: A Deeper Look

Objectives

In this chapter, you'll:

- Understand text processing.
- Use string methods.
- Format string content.
- Concatenate and repeat strings.
- Strip whitespace from the ends of strings.
- Change characters from lowercase to uppercase and vice versa.
- Compare strings with the comparison operators.
- Search strings for substrings and replace substrings.
- Split strings into tokens.
- Concatenate strings into a single string with a specified separator between items.
- Create and use regular expressions to match patterns in strings, replace substrings and validate data.
- Use regular expression metacharacters, quantifiers, character classes and grouping.
- Understand how critical string manipulations are to natural language processing.
- Understand the data science terms data munging, data wrangling and data cleaning, and use regular expressions to munge data into preferred formats.

.1 Introduction

.2 Formatting Strings

.2.1 Presentation Types

.2.2 Field Widths and Alignment

.2.3 Numeric Formatting

.2.4 String's `format` Method

.3 Concatenating and Repeating Strings

.4 Stripping Whitespace from Strings

.5 Changing Character Case

.6 Comparison Operators for Strings

.7 Searching for Substrings

.8 Replacing Substrings

.9 Splitting and Joining Strings

.10 Characters and Character-Testing Methods

.11 Raw Strings

.12 Introduction to Regular Expressions

.12.1 `re` Module and Function `fullmatch`

.12.2 Replacing Substrings and Splitting Strings

.12.3 Other Search Functions; Accessing Matches

.13 Intro to Data Science: Pandas, Regular Expressions and Data Munging

.14 Wrap-Up

.1 INTRODUCTION

We've introduced strings, basic string formatting and several string operators and methods. You saw that strings support many of the same sequence operations as lists and tuples, and that strings, like tuples, are immutable. Now, we take a deeper look at strings and introduce regular expressions and the `re` module, which we'll use to match patterns ¹ in text. Regular expressions are particularly important in today's data rich applications. The capabilities presented here will help you prepare for the “Natural language Processing (NLP)” chapter and other key data science chapters. In the NLP chapter, we'll look at other ways to have computers manipulate and even “understand” text. The table below shows many string-processing and NLP-related applications. In the Intro to Data Science section, we briefly introduce data cleaning/munging/wrangling with Pandas `Series` and `DataFrames`.

¹ Well see in the data science case study chapters that searching for patterns in text is a crucial part of machine learning.

String and NLP applications

Anagrams

Automated
grading of
written
homework

Inter-language translation

Automated
teaching systems

Legal document
preparation

Spam classification

Categorizing
articles

Monitoring social media
posts

Speech-to-text engines

Chatbots

Natural language
understanding-

Steganography

Compilers and

Opinion analysis

Text editors

interpreters	Page-composition software	Text-to-speech engines
Creative writing	Palindromes	Web scraping
Cryptography	Parts-of-speech tagging	Who authored Shakespeare's works?
Document classification	Project Gutenberg free books	Word clouds
Document similarity	Reading books, articles, documentation and absorbing knowledge	Word games
Document summarization	Search engines	Writing medical diagnoses from x-rays, scans, blood tests
Electronic book readers	Sentiment analysis	and many more
Fraud detection		
Grammar checkers		

8.2 FORMATTING STRINGS

Proper text formatting makes data easier to read and understand. Here, we present many text-formatting capabilities.

8.2.1 Presentation Types

You've seen basic string formatting with f-strings. When you specify a placeholder for a value in an f-string, Python assumes the value should be displayed as a string unless you specify another type. In some cases, the type is required. For example, let's format the `float` value `17.489` rounded to the hundredths position:

```
In [1]: f'{17.489:.2f}'
Out[1]: '17.49'
```

Python supports precision *only* for floating-point and `Decimal` values. Formatting is *type dependent*—if you try to use `.2f` to format a string like `'hello'`, a `ValueError` occurs. So the **presentation type** `f` in the *format specifier* `.2f` is required. It indicates what type is being formatted so Python can determine whether the other formatting information is allowed for that type. Here, we show some common presentation types. You can view the complete list at

<https://docs.python.org/3/library/string.html#formatspec>

Integers

The **d presentation type** formats integer values as strings:

```
In [2]: f'{10:d}'  
Out[2]: '10'
```

There also are integer presentation types (`b`, `o` and `x` or `X`) that format integers using the binary, octal or hexadecimal number systems. ²

² See the online appendix Number Systems for information about the binary, octal and hexadecimal number systems.

Characters

The **c presentation type** formats an integer character code as the corresponding character:

[lick here to view code image](#)

```
In [3]: f'{65:c} {97:c}'  
Out[3]: 'A a'
```

Strings

The **s presentation type** is the default. If you specify `s` explicitly, the value to format must be a variable that references a string, an expression that produces a string or a string literal, as in the first placeholder below. If you do not specify a presentation type, as in the second placeholder below, non-string values like the integer `7` are converted to strings:

[lick here to view code image](#)

```
In [4]: f'{"hello":s} {7}'  
Out[4]: 'hello 7'
```

In this snippet, "hello" is enclosed in double quotes. Recall that you cannot place single quotes inside a single-quoted string.

Floating-Point and Decimal Values

You've used the `f` presentation type to format floating-point and `Decimal` values. For extremely large and small values of these types, **Exponential (scientific) notation** can be used to format the values more compactly. Let's show the difference between `f` and `e` for a large value, each with three digits of precision to the right of the decimal point:

[lick here to view code image](#)

```
In [5]: from decimal import Decimal  
  
In [6]: f'{Decimal("10000000000000000000000000.0"):.3f}'  
Out[6]: '10000000000000000000000000.000'  
  
In [7]: f'{Decimal("10000000000000000000000000.0"):.3e}'  
Out[7]: '1.000e+25'
```

For the **e presentation type** in snippet [5], the formatted value `1.000e+25` is equivalent to

$$1.000 \times 10^{25}$$

If you prefer a capital `E` for the exponent, use the **E presentation type** rather than `e`.

8.2.2 Field Widths and Alignment

Previously you used *field widths* to format text in a specified number of character positions. By default, Python *right-aligns* numbers and *left-aligns* other values such as strings—we enclose the results below in brackets (`[]`) so you can see how the values align in the field:

[lick here to view code image](#)

```
In [1]: f'[{27:10d}]'
```

```
Out[1]: '[          27]'
```

```
In [2]: f'[{3.5:10f}]'
```

```
Out[2]: '[  3.500000]'
```



```
In [3]: f'["hello":10]'
```

```
Out[3]: '[hello      ]'
```

Snippet [2] shows that Python formats `float` values with six digits of precision to the right of the decimal point by default. For values that have fewer characters than the field width, the remaining character positions are filled with spaces. Values with more characters than the field width use as many character positions as they need.

Explicitly Specifying Left and Right Alignment in a Field

Recall that you can specify left and right alignment with `<` and `>`:

[lick here to view code image](#)

```
In [4]: f'[{27:<15d}]'
```

```
Out[4]: '[27                ]'
```



```
In [5]: f'[{3.5:<15f}]'
```

```
Out[5]: '[3.500000          ]'
```



```
In [6]: f'["hello":>15]'
```

```
Out[6]: '[          hello]'
```

Centering a Value in a Field

In addition, you can *center* values:

[lick here to view code image](#)

```
In [7]: f'[{27:^7d}]'
```

```
Out[7]: '[  27  ]'
```



```
In [8]: f'[{3.5:^7.1f}]'
```

```
Out[8]: '[  3.5  ]'
```



```
In [9]: f'["hello":^7]'
```

```
Out[9]: '[ hello ]'
```

Centering attempts to spread the remaining unoccupied character positions equally to the left and right of the formatted value. Python places the extra space to the right if an

odd number of character positions remain.

8.2.3 Numeric Formatting

There are a variety of numeric formatting capabilities.

Formatting Positive Numbers with Signs

Sometimes it's desirable to force the sign on a positive number:

```
In [1]: f'[{27:+10d}] '  
Out[1]: '[          +27] '
```

The `+` before the field width specifies that a positive number should be preceded by a `+`. A negative number always starts with a `-`. To fill the remaining characters of the field with `0`s rather than spaces, place a `0` before the field width (and *after* the `+` if there is one):

```
In [2]: f'[{27:+010d}] '  
Out[2]: '[+000000027] '
```

Using a Space Where a `+` Sign Would Appear in a Positive Value

A space indicates that positive numbers should show a space character in the sign position. This is useful for aligning positive and negative values for display purposes:

[lick here to view code image](#)

```
In [3]: print(f'{27:d}\n{27: d}\n{-27: d}')
```

27
 27
-27

Note that the two numbers with a space in their format specifiers align. If a field width is specified, the space should appear *before* the field width.

Grouping Digits

You can format numbers with **thousands separators** by using a **comma (,)**, as follows:

```
In [4]: f'{12345678:,d} '
```

```
Out[4]: '12,345,678'

In [5]: f'{123456.78:,.2f}'
Out[5]: '123,456.78'
```

8.2.4 String's `format` Method

Python's f-strings were added to the language in version 3.6. Before that, formatting was performed with the string method `format`. In fact, f-string formatting is based on the `format` method's capabilities. We show you the `format` method here because you'll encounter it in code written prior to Python 3.6. You'll often see the `format` method in the Python documentation and in the many Python books and articles written before f-strings were introduced. However, we recommend using the newer f-string formatting that we've presented to this point.

You call method `format` on a *format string* containing curly brace (`{ }`) *placeholders*, possibly with format specifiers. You pass to the method the values to be formatted. Let's format the `float` value `17.489` rounded to the hundredths position:

[lick here to view code image](#)

```
In [1]: '{:.2f}'.format(17.489)
Out[1]: '17.49'
```

In a placeholder, if there's a format specifier, you precede it by a colon (`:`), as in f-strings. The result of the `format` call is a new string containing the formatted results.

Multiple Placeholders

A format string may contain multiple placeholders, in which case the `format` method's arguments correspond to the placeholders from left to right:

[lick here to view code image](#)

```
In [2]: '{} {}'.format('Amanda', 'Cyan')
Out[2]: 'Amanda Cyan'
```

Referencing Arguments By Position Number

The format string can reference specific arguments by their position in the `format` method's argument list, starting with position 0:

[lick here to view code image](#)

```
In [3]: '{0} {0} {1}'.format('Happy', 'Birthday')
Out[3]: 'Happy Happy Birthday'
```

Note that we used the position number 0 ('Happy') twice—you can reference each argument as often as you like and in any order.

Referencing Keyword Arguments

You can reference keyword arguments by their keys in the placeholders:

[lick here to view code image](#)

```
In [4]: '{first} {last}'.format(first='Amanda', last='Gray')
Out[4]: 'Amanda Gray'

In [5]: '{last} {first}'.format(first='Amanda', last='Gray')
Out[5]: 'Gray Amanda'
```

8.3 CONCATENATING AND REPEATING STRINGS

In earlier chapters, we used the + operator to concatenate strings and the * operator to repeat strings. You also can perform these operations with augmented assignments. Strings are immutable, so each operation assigns a new string object to the variable:

[lick here to view code image](#)

```
In [1]: s1 = 'happy'

In [2]: s2 = 'birthday'

In [3]: s1 += ' ' + s2

In [4]: s1
Out[4]: 'happy birthday'

In [5]: symbol = '>'

In [6]: symbol *= 5

In [7]: symbol
Out[7]: '>>>>>'
```

8.4 STRIPPING WHITESPACE FROM STRINGS

There are several string methods for removing whitespace from the ends of a string. Each returns a new string leaving the original unmodified. Strings are immutable, so each method that appears to modify a string returns a new one.

Removing Leading and Trailing Whitespace

Let's use string method **strip** to remove the leading and trailing whitespace from a string:

[lick here to view code image](#)

```
In [1]: sentence = '\t \n This is a   test string. \t\t \n'

In [2]: sentence.strip()
Out[2]: 'This is a test string.'
```

Removing Leading Whitespace

Method **lstrip** removes only leading whitespace:

[lick here to view code image](#)

```
In [3]: sentence.lstrip()
Out[3]: 'This is a test string. \t\t \n'
```

Removing Trailing Whitespace

Method **rstrip** removes only trailing whitespace:

[lick here to view code image](#)

```
In [4]: sentence.rstrip()
Out[4]: '\t \n This is a test string.'
```

As the outputs demonstrate, these methods remove all kinds of whitespace, including spaces, newlines and tabs.

8.5 CHANGING CHARACTER CASE

In earlier chapters, you used string methods `lower` and `upper` to convert strings to all

lowercase or all uppercase letters. You also can change a string's capitalization with methods `capitalize` and `title`.

Capitalizing Only a String's First Character

Method `capitalize` copies the original string and returns a new string with only the first letter capitalized (this is sometimes called *sentence capitalization*):

[lick here to view code image](#)

```
In [1]: 'happy birthday'.capitalize()  
Out[1]: 'Happy birthday'
```

Capitalizing the First Character of Every Word in a String

Method `title` copies the original string and returns a new string with only the first character of each word capitalized (this is sometimes called *book-title capitalization*):

[lick here to view code image](#)

```
In [2]: 'strings: a deeper look'.title()  
Out[2]: 'Strings: A Deeper Look'
```

8.6 COMPARISON OPERATORS FOR STRINGS

Strings may be compared with the comparison operators. Recall that strings are compared based on their underlying integer numeric values. So uppercase letters compare as less than lowercase letters because uppercase letters have lower integer values. For example, 'A' is 65 and 'a' is 97. You've seen that you can check character codes with `ord`:

[lick here to view code image](#)

```
In [1]: print(f'A: {ord("A")}; a: {ord("a")}')  
A: 65; a: 97
```

Let's compare the strings 'Orange' and 'orange' using the comparison operators:

[lick here to view code image](#)

```
In [2]: 'Orange' == 'orange'
```

```
Out[2]: False

In [3]: 'Orange' != 'orange'
Out[3]: True

In [4]: 'Orange' < 'orange'
Out[4]: True

In [5]: 'Orange' <= 'orange'
Out[5]: True

In [6]: 'Orange' > 'orange'
Out[6]: False

In [7]: 'Orange' >= 'orange'
Out[7]: False
```

8.7 SEARCHING FOR SUBSTRINGS

You can search in a string for one or more adjacent characters—known as a *substring*—to count the number of occurrences, determine whether a string contains a substring, or determine the index at which a substring resides in a string. Each method shown in this section compares characters lexicographically using their underlying numeric values.

Counting Occurrences

String method `count` returns the number of times its argument occurs in the string on which the method is called:

[lick here to view code image](#)

```
In [1]: sentence = 'to be or not to be that is the question'

In [2]: sentence.count('to')
Out[2]: 2
```

If you specify as the second argument a *start_index*, `count` searches only the slice `string[start_index:]`—that is, from *start_index* through end of the string:

[lick here to view code image](#)

```
In [3]: sentence.count('to', 12)
Out[3]: 1
```

If you specify as the second and third arguments the *start_index* and *end_index*, `count` searches only the slice *string* [*start_index*:*end_index*] —that is, from *start_index* up to, but not including, *end_index*:

[lick here to view code image](#)

```
In [4]: sentence.count('that', 12, 25)
Out[4]: 1
```

Like `count`, each of the other string methods presented in this section has *start_index* and *end_index* arguments for searching only a slice of the original string.

Locating a Substring in a String

String method **`index`** searches for a substring within a string and returns the first index at which the substring is found; otherwise, a `ValueError` occurs:

[lick here to view code image](#)

```
In [5]: sentence.index('be')
Out[5]: 3
```

String method **`rindex`** performs the same operation as `index`, but searches from the end of the string and returns the *last* index at which the substring is found; otherwise, a `Value-Error` occurs:

[lick here to view code image](#)

```
In [6]: sentence.rindex('be')
Out[6]: 16
```

String methods **`find`** and **`rfind`** perform the same tasks as `index` and `rindex` but, if the substring is not found, return `-1` rather than causing a `Value-Error`.

Determining Whether a String Contains a Substring

If you need to know only whether a string contains a substring, use operator `in` or `not in`:

[lick here to view code image](#)

```
In [7]: 'that' in sentence
Out[7]: True

In [8]: 'THAT' in sentence
Out[8]: False

In [9]: 'THAT' not in sentence
Out[9]: True
```

Locating a Substring at the Beginning or End of a String

String methods **startswith** and **endswith** return `True` if the string starts with or ends with a specified substring:

[lick here to view code image](#)

```
In [10]: sentence.startswith('to')
Out[10]: True

In [11]: sentence.startswith('be')
Out[11]: False

In [12]: sentence.endswith('question')
Out[12]: True

In [13]: sentence.endswith('quest')
Out[13]: False
```

8.8 REPLACING SUBSTRINGS

A common text manipulation is to locate a substring and replace its value. Method **replace** takes two substrings. It searches a string for the substring in its first argument and replaces *each* occurrence with the substring in its second argument. The method returns a new string containing the results. Let's replace tab characters with commas:

[lick here to view code image](#)

```
In [1]: values = '1\t2\t3\t4\t5'

In [2]: values.replace('\t', ',')
Out[2]: '1,2,3,4,5'
```

Method `replace` can receive an optional third argument specifying the maximum

number of replacements to perform.

8.9 SPLITTING AND JOINING STRINGS

When you read a sentence, your brain breaks it into individual words, or **tokens**, each of which conveys meaning. Interpreters like IPython tokenize statements, breaking them into individual components such as keywords, identifiers, operators and other elements of a programming language. Tokens typically are separated by whitespace characters such as blank, tab and newline, though other characters may be used—the separators are known as **delimiters**.

Splitting Strings

We showed previously that string method `split` with *no* arguments tokenizes a string by breaking it into substrings at each whitespace character, then returns a list of tokens. To tokenize a string at a custom delimiter (such as each comma-and-space pair), specify the delimiter string (such as `,`) that `split` uses to tokenize the string:

[lick here to view code image](#)

```
In [1]: letters = 'A, B, C, D'

In [2]: letters.split(', ')
Out[2]: ['A', 'B', 'C', 'D']
```

If you provide an integer as the second argument, it specifies the maximum number of splits. The last token is the remainder of the string after the maximum number of splits:

[lick here to view code image](#)

```
In [3]: letters.split(', ', 2)
Out[3]: ['A', 'B', 'C, D']
```

There is also an **`rsplit`** method that performs the same task as `split` but processes the maximum number of splits from the end of the string toward the beginning.

Joining Strings

String method **`join`** concatenates the strings in its argument, which must be an iterable containing only string values; otherwise, a `TypeError` occurs. The separator between the concatenated items is the string on which you call `join`. The following

code creates strings containing comma-separated lists of values:

[lick here to view code image](#)

```
In [4]: letters_list = ['A', 'B', 'C', 'D']
In [5]: ','.join(letters_list)
Out[5]: 'A,B,C,D'
```

The next snippet joins the results of a list comprehension that creates a list of strings:

[lick here to view code image](#)

```
In [6]: ','.join([str(i) for i in range(10)])
Out[6]: '0,1,2,3,4,5,6,7,8,9'
```

In the “Files and Exceptions” chapter, you’ll see how to work with files that contain comma-separated values. These are known as **CSV files** and are a common format for storing data that can be loaded by spreadsheet applications like Microsoft Excel or Google Sheets. In the data science case study chapters, you’ll see that many key libraries, such as NumPy, Pandas and Seaborn, provide built-in capabilities for working with CSV data.

String Methods `partition` and `rpartition`

String method `partition` splits a string into a tuple of three strings based on the method’s *separator* argument. The three strings are

- the part of the original string before the separator,
- the separator itself, and
- the part of the string after the separator.

This might be useful for splitting more complex strings. Consider a string representing a student’s name and grades:

```
'Amanda: 89, 97, 92'
```

Let’s split the original string into the student’s name, the separator `:` and a string representing the list of grades:

[lick here to view code image](#)

```
In [7]: 'Amanda: 89, 97, 92'.partition(': ')
Out[7]: ('Amanda', ': ', '89, 97, 92')
```

To search for the separator from the end of the string instead, use method **rpartition** to split. For example, consider the following URL string:

```
' http://www.deitel.com/books/PyCDS/table_of_contents.html'
```

Let's use **rpartition** to split 'table_of_contents.html' from the rest of the URL:

[lick here to view code image](#)

```
In [8]: url = 'http://www.deitel.com/books/PyCDS/table_of_contents.html'

In [9]: rest_of_url, separator, document = url.rpartition('/')

In [10]: document
Out[10]: 'table_of_contents.html'

In [11]: rest_of_url
Out[11]: 'http://www.deitel.com/books/PyCDS'
```

String Method **splitlines**

In the “Files and Exceptions” chapter, you’ll read text from a file. If you read large amounts of text into a string, you might want to split the string into a list of lines based on newline characters. Method **splitlines** returns a list of new strings representing the lines of text split at each newline character in the original string. Recall that Python stores multiline strings with embedded `\n` characters to represent the line breaks, as shown in snippet [13]:

[lick here to view code image](#)

```
In [12]: lines = """This    is line 1
...: This is line2
...: This is    line3"""

In [13]: lines
Out[13]: 'This is line 1\nThis is line2\nThis is line3'
```

```
In [14]: lines.splitlines()
Out[14]: ['This is line 1', 'This is line2', 'This is   line3']
```

Passing `True` to `splitlines` keeps the newlines at the end of each string:

[lick here to view code image](#)

```
In [15]: lines.splitlines(True)
Out[15]: ['This is line 1\n', 'This is line2\n', 'This is   line3']
```

8.10 CHARACTERS AND CHARACTER-TESTING METHODS

Many programming languages have separate string and character types. In Python, a character is simply a one-character string.

Python provides string methods for testing whether a string matches certain characteristics. For example, string method `isdigit` returns `True` if the string on which you call the method contains only the digit characters (0–9). You might use this when validating user input that must contain only digits:

[lick here to view code image](#)

```
In [1]: '-27'.isdigit()
Out[1]: False

In [2]: '27'.isdigit()
Out[2]: True
```

and the string method `isalnum` returns `True` if the string on which you call the method is alphanumeric—that is, it contains only digits and letters:

[lick here to view code image](#)

```
In [3]: 'A9876'.isalnum()
Out[3]: True

In [4]: '123 Main Street'.isalnum()
Out[4]: False
```

The table below shows many of the character-testing methods. Each method returns

also if the condition described is not satisfied:

String Method	Description
<code>isalnum()</code>	Returns <code>True</code> if the string contains only <i>alphanumeric</i> characters (i.e., digits and letters).
<code>isalpha()</code>	Returns <code>True</code> if the string contains only <i>alphabetic</i> characters (i.e., letters).
<code>isdecimal()</code>	Returns <code>True</code> if the string contains only <i>decimal integer</i> characters (that is, base 10 integers) and does not contain a <code>+</code> or <code>-</code> sign.
<code>isdigit()</code>	Returns <code>True</code> if the string contains only digits (e.g., <code>'0'</code> , <code>'1'</code> , <code>'2'</code>).
<code>isidentifier()</code>	Returns <code>True</code> if the string represents a valid <i>identifier</i> .
<code>islower()</code>	Returns <code>True</code> if all alphabetic characters in the string are <i>lowercase</i> characters (e.g., <code>'a'</code> , <code>'b'</code> , <code>'c'</code>).
<code>isnumeric()</code>	Returns <code>True</code> if the characters in the string represent a <i>numeric value</i> without a <code>+</code> or <code>-</code> sign and without a decimal point.
<code>isspace()</code>	Returns <code>True</code> if the string contains only <i>whitespace</i> characters.

`istitle()`

Returns `True` if the first character of each word in the string is the only *uppercase* character in the word.

`isupper()`

Returns `True` if all alphabetic characters in the string are *uppercase* characters (e.g., 'A', 'B', 'C').

8.11 RAW STRINGS

Recall that backslash characters in strings introduce *escape sequences*—like `\n` for newline and `\t` for tab. So, if you wish to include a backslash in a string, you must use two back-slash characters `\\`. This makes some strings difficult to read. For example, Microsoft Windows uses backslashes to separate folder names when specifying a file's location. To represent a file's location on Windows, you might write:

[lick here to view code image](#)

```
In [1]: file_path = 'C:\\MyFolder\\MySubFolder\\MyFile.txt'

In [2]: file_path
Out[2]: 'C:\\MyFolder\\MySubFolder\\MyFile.txt'
```

For such cases, **raw strings**—preceded by the character `r`—are more convenient. They treat each backslash as a regular character, rather than the beginning of an escape sequence:

[lick here to view code image](#)

```
In [3]: file_path = r'C:\MyFolder\MySubFolder\MyFile.txt'

In [4]: file_path
Out[4]: 'C:\\MyFolder\\MySubFolder\\MyFile.txt'
```

Python converts the raw string to a regular string that still uses the two backslash characters in its internal representation, as shown in the last snippet. Raw strings can make your code more readable, particularly when using the regular expressions that we discuss in the next section. Regular expressions often contain many backslash characters.

.12 INTRODUCTION TO REGULAR EXPRESSIONS

Sometimes you'll need to recognize *patterns* in text, like phone numbers, e-mail addresses, ZIP Codes, web page addresses, Social Security numbers and more. A **regular expression** string describes a *search pattern* for *matching* characters in other strings.

Regular expressions can help you extract data from unstructured text, such as social media posts. They're also important for ensuring that data is in the correct format before you attempt to process it. ³

³ The topic of regular expressions might feel more challenging than most other Python features you've used. After mastering this subject, you'll often write more concise code than with conventional string-processing techniques, speeding the code-development process. You'll also deal with fringe cases you might not ordinarily think about, possibly avoiding subtle bugs.

Validating Data

Before working with text data, you'll often use regular expressions to *validate the data*. For example, you can check that:

- A U.S. ZIP Code consists of five digits (such as 02215) or five digits followed by a hyphen and four more digits (such as 02215-4775).
- A string last name contains only letters, spaces, apostrophes and hyphens.
- An e-mail address contains only the allowed characters in the allowed order.
- A U.S. Social Security number contains three digits, a hyphen, two digits, a hyphen and four digits, and adheres to other rules about the specific numbers that can be used in each group of digits.

You'll rarely need to create your own regular expressions for common items like these. Websites like

- <https://regex101.com>
- <http://www.regexlib.com>
- <https://www.regular-expressions.info>

and others offer repositories of existing regular expressions that you can copy and use. Many sites like these also provide interfaces in which you can test regular expressions to determine whether they'll meet your needs.

Other Uses of Regular Expressions

In addition to validating data, regular expressions often are used to:

- Extract data from text (sometimes known as *scraping*)—For example, locating all URLs in a web page. [You might prefer tools like BeautifulSoup, XPath and lxml.]
- Clean data—For example, removing data that's not required, removing duplicate data, handling incomplete data, fixing typos, ensuring consistent data formats, dealing with outliers and more.
- Transform data into other formats—For example, reformatting data that was collected as tab-separated or space-separated values into comma-separated values (CSV) for an application that requires data to be in CSV format.

8.12.1 `re` Module and Function `fullmatch`

To use regular expressions, import the Python Standard Library's `re` module:

```
In [1]: import re
```

One of the simplest regular expression functions is `fullmatch`, which checks whether the *entire* string in its second argument matches the pattern in its first argument.

Matching Literal Characters

Let's begin by matching *literal characters*—that is, characters that match themselves:

[lick here to view code image](#)

```
In [2]: pattern = '02215'

In [3]: 'Match' if re.fullmatch(pattern, '02215') else 'No match'
Out[3]: 'Match'

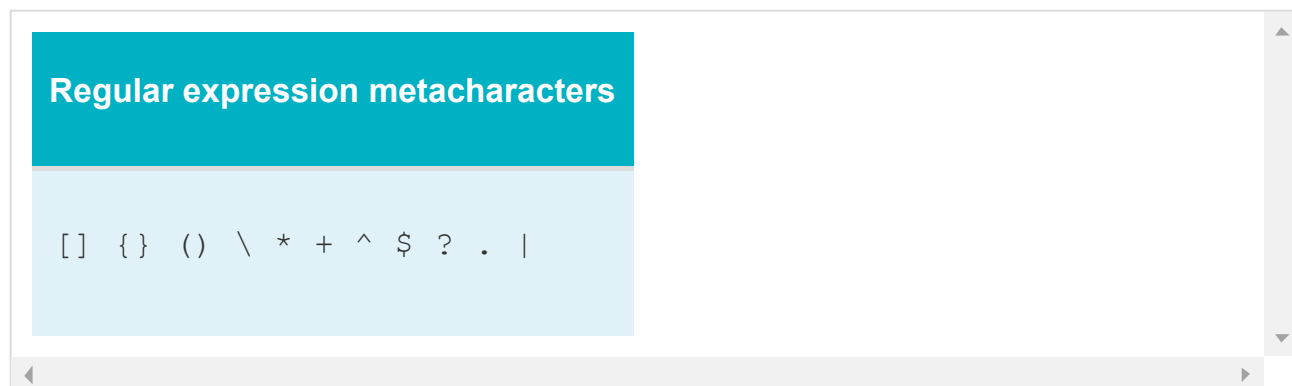
In [4]: 'Match' if re.fullmatch(pattern, '51220') else 'No match'
Out[4]: 'No match'
```

The function's first argument is the regular expression pattern to match. Any string can be a regular expression. The variable `pattern`'s value, `'02215'`, contains only *literal digits* that match *themselves* in the specified order. The second argument is the string that should entirely match the pattern.

If the second argument matches the pattern in the first argument, `fullmatch` returns an object containing the matching text, which evaluates to `True`. We'll say more about this object later. In snippet [4], even though the second argument contains the *same digits* as the regular expression, they're in a *different* order. So there's no match, and `fullmatch` returns `None`, which evaluates to `False`.

Metacharacters, Character Classes and Quantifiers

Regular expressions typically contain various special symbols called **metacharacters**, which are shown in the table below:



The **\ metacharacter** begins each of the predefined **character classes**, each matching a specific set of characters. Let's validate a five-digit ZIP Code:

[lick here to view code image](#)

```
In [5]: 'Valid' if re.fullmatch(r'\d{5}', '02215') else 'Invalid'
Out[5]: 'Valid'

In [6]: 'Valid' if re.fullmatch(r'\d{5}', '9876') else 'Invalid'
Out[6]: 'Invalid'
```

In the regular expression `\d{5}`, **\d** is a character class representing a digit (0–9). A character class is a *regular expression escape sequence* that matches *one* character. To match more than one, follow the character class with a **quantifier**. The quantifier `{5}` repeats `\d` five times, as if we had written `\d\d\d\d\d`, to match five consecutive digits. In snippet [6], `fullmatch` returns `None` because `'9876'` contains only four consecutive digit characters.

Other Predefined Character Classes

The table below shows some common predefined character classes and the groups of characters they match. To match any metacharacter as its *literal* value, precede it by a backslash (\). For example, `\\` matches a backslash (\) and `\$` matches a dollar sign (\$).

Character class	Matches
<code>\d</code>	Any digit (0–9).
<code>\D</code>	Any character that is <i>not</i> a digit.
<code>\s</code>	Any whitespace character (such as spaces, tabs and newlines).
<code>\S</code>	Any character that is <i>not</i> a whitespace character.
<code>\w</code>	Any word character (also called an alphanumeric character)—that is, any uppercase or lowercase letter, any digit or an underscore
<code>\W</code>	Any character that is <i>not</i> a word character.

Custom Character Classes

Square brackets, `[]`, define a **custom character class** that matches a *single* character. For example, `[aeiou]` matches a lowercase vowel, `[A-Z]` matches an uppercase letter, `[a-z]` matches a lowercase letter and `[a-zA-Z]` matches any lowercase or uppercase letter.

Let's validate a simple first name with no spaces or punctuation. We'll ensure that it begins with an uppercase letter (A–Z) followed by any number of lowercase letters (a–z):

[lick here to view code image](#)

```
In [7]: 'Valid' if re.fullmatch('[A-Z][a-z]*', 'Wally') else 'Invalid'
Out[7]: 'Valid'

In [8]: 'Valid' if re.fullmatch('[A-Z][a-z]*', 'eva') else 'Invalid'
Out[8]: 'Invalid'
```

A first name might contain many letters. The *** quantifier** matches *zero or more occurrences* of the subexpression to its left (in this case, `[a-z]`). So `[A-Z][a-z]*` matches an uppercase letter followed by *zero or more* lowercase letters, such as 'Amanda', 'Bo' or even 'E'.

When a custom character class starts with a **caret (^)**, the class matches any character that's *not* specified. So `[^a-z]` matches any character that's *not* a lowercase letter:

[lick here to view code image](#)

```
In [9]: 'Match' if re.fullmatch('[^a-z]', 'A') else 'No match'
Out[9]: 'Match'

In [10]: 'Match' if re.fullmatch('[^a-z]', 'a') else 'No match'
Out[10]: 'No match'
```

Metacharacters in a custom character class are treated as literal characters—that is, the characters themselves. So `[*+&]` matches a *single* *, + or & character:

[lick here to view code image](#)

```
In [11]: 'Match' if re.fullmatch('[*+&]', '*') else 'No match'
Out[11]: 'Match'

In [12]: 'Match' if re.fullmatch('[*+&]', '!!') else 'No match'
Out[12]: 'No match'
```

* vs. + Quantifier

If you want to require *at least one* lowercase letter in a first name, you can replace the *

quantifier in snippet [7] with **+**, which matches *at least one occurrence* of a subexpression:

[lick here to view code image](#)

```
In [13]: 'Valid' if re.fullmatch('[A-Z][a-z]+', 'Wally') else 'Invalid'
Out[13]: 'Valid'

In [14]: 'Valid' if re.fullmatch('[A-Z][a-z]+', 'E') else 'Invalid'
Out[14]: 'Invalid'
```

Both ***** and **+** are **greedy**—they match as many characters as possible. So the regular expression `[A-Z][a-z]+` matches `'Al'`, `'Eva'`, `'Samantha'`, `'Benjamin'` and any other words that begin with a capital letter followed at least one lowercase letter.

Other Quantifiers

The **?** **quantifier** matches *zero or one occurrences* of a subexpression:

[lick here to view code image](#)

```
In [15]: 'Match' if re.fullmatch('labell?ed', 'labelled') else 'No match'
Out[15]: 'Match'

In [16]: 'Match' if re.fullmatch('labell?ed', 'labeled') else 'No match'
Out[16]: 'Match'

In [17]: 'Match' if re.fullmatch('labell?ed', 'labellled') else 'No match'
Out[17]: 'No match'
```

The regular expression `labell?ed` matches `labelled` (the U.K. English spelling) and `labeled` (the U.S. English spelling), but not the misspelled word `labellled`. In each snippet above, the first five literal characters in the regular expression (`label`) match the first five characters of the second arguments. Then `l?` indicates that there can be *zero or one more* `l` characters before the remaining literal `ed` characters.

You can match *at least n occurrences* of a subexpression with the **{ n , }** **quantifier**. The following regular expression matches strings containing *at least* three digits:

[lick here to view code image](#)


```
In [18]: 'Match' if re.fullmatch(r'\d{3,}', '123') else 'No match'
Out[18]: 'Match'

In [19]: 'Match' if re.fullmatch(r'\d{3,}', '1234567890') else 'No match'
Out[19]: 'Match'

In [20]: 'Match' if re.fullmatch(r'\d{3,}', '12') else 'No match'
Out[20]: 'No match'
```

You can match *between n and m (inclusive) occurrences* of a subexpression with the **{ n,m } quantifier**. The following regular expression matches strings containing 3 to 6 digits:

[lick here to view code image](#)

```
In [21]: 'Match' if re.fullmatch(r'\d{3,6}', '123') else 'No match'
Out[21]: 'Match'

In [22]: 'Match' if re.fullmatch(r'\d{3,6}', '123456') else 'No match'
Out[22]: 'Match'

In [23]: 'Match' if re.fullmatch(r'\d{3,6}', '1234567') else 'No match'
Out[23]: 'No match'

In [24]: 'Match' if re.fullmatch(r'\d{3,6}', '12') else 'No match'
Out[24]: 'No match'
```

8.12.2 Replacing Substrings and Splitting Strings

The `re` module provides function `sub` for replacing patterns in a string, and function `split` for breaking a string into pieces, based on patterns.

Function `sub`—Replacing Patterns

By default, the `re` module's **sub function** replaces *all* occurrences of a pattern with the replacement text you specify. Let's convert a tab-delimited string to comma-delimited:

[lick here to view code image](#)

```
In [1]: import re

In [2]: re.sub(r'\t', ',', '1\t2\t3\t4')
```

```
Out[2]: '1, 2, 3, 4'
```

The `sub` function receives three required arguments:

- the *pattern to match* (the tab character `'\t'`)
- the *replacement text* (`','`) and
- the *string to be searched* (`'1\t2\t3\t4'`)

and returns a new string. The keyword argument `count` can be used to specify the maximum number of replacements:

[lick here to view code image](#)

```
In [3]: re.sub(r'\t', ',', '1\t2\t3\t4', count=2)
Out[3]: '1, 2, 3\t4'
```

Function `split`

The **`split` function** *tokenizes* a string, using a regular expression to specify the *delimiter*, and returns a list of strings. Let's tokenize a string by splitting it at any comma that's followed by 0 or more whitespace characters—`\s` is the whitespace character class and `*` indicates *zero or more* occurrences of the preceding subexpression:

[lick here to view code image](#)

```
In [4]: re.split(r',\s*', '1, 2, 3,4, 5,6,7,8')
Out[4]: ['1', '2', '3', '4', '5', '6', '7', '8']
```

Use the keyword argument `maxsplit` to specify the maximum number of splits:

[lick here to view code image](#)

```
In [5]: re.split(r',\s*', '1, 2, 3,4, 5,6,7,8', maxsplit=3)
Out[5]: ['1', '2', '3', '4, 5,6,7,8']
```

In this case, after the 3 splits, the fourth string contains the rest of the original string.

8.12.3 Other Search Functions; Accessing Matches

Earlier we used the `fullmatch` function to determine whether an *entire* string matched a regular expression. There are several other searching functions. Here, we discuss the `search`, `match`, `findall` and `finditer` functions, and show how to access the matching substrings.

Function `search`—Finding the First Match Anywhere in a String

Function `search` looks in a string for the *first* occurrence of a substring that matches a regular expression and returns a **match object** (of type `SRE_Match`) that contains the matching substring. The match object's `group` method returns that substring:

[lick here to view code image](#)

```
In [1]: import re

In [2]: result = re.search('Python', 'Python is fun')

In [3]: result.group() if result else 'not found'
Out[3]: 'Python'
```

Function `search` returns `None` if the string does *not* contain the pattern:

[lick here to view code image](#)

```
In [4]: result2 = re.search('fun!', 'Python is fun')

In [5]: result2.group() if result2 else 'not found'
Out[5]: 'not found'
```

You can search for a match only at the *beginning* of a string with function `match`.

Ignoring Case with the Optional `flags` Keyword Argument

Many `re` module functions receive an optional `flags` keyword argument that changes how regular expressions are matched. For example, matches are *case sensitive* by default, but by using the `re` module's `IGNORECASE` constant, you can perform a *case-insensitive* search:

[lick here to view code image](#)

```
In [6]: result3 = re.search('Sam', 'SAM WHITE', flags=re.IGNORECASE)
```

```
In [7]: result3.group() if result3 else 'not found'
Out[7]: 'SAM'
```

Here, 'SAM' matches the pattern 'Sam' because both have the same letters, even though 'SAM' contains only uppercase letters.

Metacharacters That Restrict Matches to the Beginning or End of a String

The **^ metacharacter** at the beginning of a regular expression (and not inside square brackets) is an anchor indicating that the expression matches only the *beginning* of a string:

[lick here to view code image](#)

```
In [8]: result = re.search('^Python', 'Python is fun')

In [9]: result.group() if result else 'not found'
Out[9]: 'Python'

In [10]: result = re.search('^fun', 'Python is fun')

In [11]: result.group() if result else 'not found'
Out[11]: 'not found'
```

Similarly, the **\$ metacharacter** at the end of a regular expression is an anchor indicating that the expression matches only the *end* of a string:

[lick here to view code image](#)

```
In [12]: result = re.search('Python$', 'Python is fun')

In [13]: result.group() if result else 'not found'
Out[13]: 'not found'

In [14]: result = re.search('fun$', 'Python is fun')

In [15]: result.group() if result else 'not found'
Out[15]: 'fun'
```

Function `findall` and `finditer`—Finding All Matches in a String

Function **`findall`** finds *every* matching substring in a string and returns a list of the matching substrings. Let's extract all the U.S. phone numbers from a string. For simplicity we'll assume that U.S. phone numbers have the form ###-###-####:

[lick here to view code image](#)

```
In [16]: contact = 'Wally White,    Home: 555-555-1234, Work: 555-555-4321'

In [17]: re.findall(r'\d{3}-\d{3}-\d{4}',    contact)
Out[17]: ['555-555-1234', '555-555-4321']
```

Function **finditer** works like `findall`, but returns a lazy *iterable* of match objects. For large numbers of matches, using `finditer` can save memory because it returns one match at a time, whereas `findall` returns all the matches at once:

[lick here to view code image](#)

```
In [18]: for phone in re.finditer(r'\d{3}-\d{3}-\d{4}',    contact):
...:     print(phone.group())
...:
555-555-1234
555-555-4321
```

Capturing Substrings in a Match

You can use **parentheses metacharacters**—(and)—to capture substrings in a match. For example, let's capture as separate substrings the name and e-mail address in the string `text`:

[lick here to view code image](#)

```
In [19]: text = 'Charlie Cyan,    e-mail: demo1@deitel.com'

In [20]: pattern = r'([A-Z][a-z]+    [A-Z][a-z]+), e-mail: (\w+@\w+\.\w{3})

In [21]: result = re.search(pattern, text)
```

The regular expression specifies two substrings to capture, each denoted by the metacharacters (and). These metacharacters do *not* affect whether the pattern is found in the string `text`—the match function returns a match object *only* if the *entire* pattern is found in the string `text`.

Let's consider the regular expression:

- `'([A-Z][a-z]+ [A-Z][a-z]+)'` matches two words separated by a space. Each

word must have an initial capital letter.

- `' , e-mail: '` contains literal characters that match themselves.
- `(\w+@\w+\.\w{3})` matches a *simple* e-mail address consisting of one or more alphanumeric characters `(\w+)`, the `@` character, one or more alphanumeric characters `(\w+)`, a dot `(\.)` and three alphanumeric characters `(\w{3})`. We preceded the dot with `\` because a dot `(.)` is a regular expression metacharacter that matches one character.

The match object's **groups** method returns a tuple of the captured substrings:

[lick here to view code image](#)

```
In [22]: result.groups()
Out[22]: ('Charlie Cyan', 'demo1@deitel.com')
```

The match object's `group` method returns the *entire* match as a single string:

[lick here to view code image](#)

```
In [23]: result.group()
Out[23]: 'Charlie Cyan, e-mail: demo1@deitel.com'
```

You can access each captured substring by passing an integer to the `group` method. The captured substrings are *numbered from* 1 (unlike list indices, which start at 0):

[lick here to view code image](#)

```
In [24]: result.group(1)
Out[24]: 'Charlie Cyan'

In [25]: result.group(2)
Out[25]: 'demo1@deitel.com'
```

8.13 INTRO TO DATA SCIENCE: PANDAS, REGULAR EXPRESSIONS AND DATA MUNGING

Data does not always come in forms ready for analysis. It could, for example, be in the wrong format, incorrect or even missing. Industry experience has shown that data

cientists can spend as much as 75% of their time preparing data before they begin their studies. Preparing data for analysis is called **data munging** or **data wrangling**. These are synonyms—from this point forward, we'll say data munging.

Two of the most important steps in data munging are *data cleaning* and *transforming data* into the optimal formats for your database systems and analytics software. Some common data cleaning examples are:

- deleting observations with missing values,
- substituting reasonable values for missing values,
- deleting observations with bad values,
- substituting reasonable values for bad values,
- tossing outliers (although sometimes you'll want to keep them),
- duplicate elimination (although sometimes duplicates are valid),
- dealing with inconsistent data,
- and more.

You're probably already thinking that data cleaning is a difficult and messy process where you could easily make bad decisions that would negatively impact your results. This is correct. When you get to the data science case studies in the later chapters, you'll see that data science is more of an **empirical science**, like medicine, and less of a theoretical science, like theoretical physics. Empirical sciences base their conclusions on observations and experience. For example, many medicines that effectively solve medical problems today were developed by observing the effects that early versions of these medicines had on lab animals and eventually humans, and gradually refining ingredients and dosages. The actions data scientists take can vary per project, be based on the quality and nature of the data and be affected by evolving organization and professional standards.

Some common data transformations include:

- removing unnecessary data and *features* (we'll say more about features in the data science case studies),

- combining related features,
- sampling data to obtain a representative subset (we'll see in the data science case studies that *random sampling* is particularly effective for this and we'll say why),
- standardizing data formats,
- grouping data,
- and more.

It's always wise to hold onto your original data. We'll show simple examples of cleaning and transforming data in the context of Pandas `Series` and `DataFrames`.

Cleaning Your Data

Bad data values and missing values can significantly impact data analysis. Some data scientists advise against any attempts to insert “reasonable values.” Instead, they advocate clearly marking missing data and leaving it up to the data analytics package to handle the issue. Others offer strong cautions. ⁴

⁴ This footnote was abstracted from a comment sent to us July 20, 2018 by one of the books reviewers, Dr. Alison Sanchez of the University of San Diego School of Business. She commented: Be cautious when mentioning 'substituting reasonable values' for missing or bad values. A stern warning: 'Substituting' values that increase statistical significance or give more 'reasonable' or 'better' results is not permitted. 'Substituting' data should not turn into 'fudging' data. The first rule readers should learn is not to eliminate or change values that contradict their hypotheses. 'Substituting reasonable values' does not mean readers should feel free to change values to get the results they want.

Let's consider a hospital that records patients' temperatures (and probably other vital signs) four times per day. Assume that the data consists of a name and four `float` values, such as

```
['Brown, Sue', 98.6, 98.4, 98.7, 0.0]
```

The preceding patient's first three recorded temperatures are 99.7, 98.4 and 98.7. The last temperature was missing and recorded as 0.0, perhaps because the sensor malfunctioned. The average of the first three values is 98.57, which is close to normal.

However, if you calculate the average temperature *including* the missing value for which 0.0 was substituted, the average is only 73.93, clearly a questionable result. Certainly, doctors would not want to take drastic remedial action on this patient—it’s crucial to “get the data right.”

One common way to clean the data is to substitute a *reasonable* value for the missing temperature, such as the average of the patient’s other readings. Had we done that above, then the patient’s average temperature would remain 98.57—a much more likely average temperature, based on the other readings.

Data Validation

Let’s begin by creating a `Series` of five-digit ZIP Codes from a dictionary of city-name/five-digit-ZIP-Code key–value pairs. We intentionally entered an invalid ZIP Code for Miami:

[lick here to view code image](#)

```
In [1]: import pandas as pd

In [2]: zips = pd.Series({'Boston': '02215', 'Miami': '3310'})

In [3]: zips
Out[3]:
Boston      02215
Miami       3310
dtype: object
```

Though `zips` looks like a two-dimensional array, it’s actually one-dimensional. The “second column” represents the `Series`’ ZIP Code *values* (from the dictionary’s values), and the “first column” represents their *indices* (from the dictionary’s keys).

We can use regular expressions with Pandas to validate data. The **`str` attribute** of a `Series` provides string-processing and various regular expression methods. Let’s use the `str` attribute’s **`match` method** to check whether each ZIP Code is valid:

[lick here to view code image](#)

```
In [4]: zips.str.match(r'\d{5}')
Out[4]:
Boston      True
Miami      False
dtype: bool
```

Method `match` applies the regular expression `\d{5}` to *each* `Series` element, attempting to ensure that the element is comprised of exactly five digits. You do not need to loop explicitly through all the ZIP Codes—`match` does this for you. This is another example of functional-style programming with internal rather than external iteration. The method returns a new `Series` containing `True` for each valid element. In this case, the ZIP Code for Miami did *not* match, so its element is `False`.

There are several ways to deal with invalid data. One is to catch it at its source and interact with the source to correct the value. That's not always possible. For example, the data could be coming from high-speed sensors in the Internet of Things. In that case, we would not be able to correct it at the source, so we could apply data cleaning techniques. In the case of the bad Miami ZIP Code of 3310, we might look for Miami ZIP Codes beginning with 3310. There are two—33101 and 33109—and we could pick one of those.

Sometimes, rather than matching an *entire* value to a pattern, you'll want to know whether a value contains a *substring* that matches the pattern. In this case, use method **`contains-`** instead of `match`. Let's create a `Series` of strings, each containing a U.S. city, state and ZIP Code, then determine whether each string contains a substring matching the pattern `' [A-Z]{2} '` (a space, followed by two uppercase letters, followed by a space):

[lick here to view code image](#)

```
In [5]: cities = pd.Series(['Boston, MA 02215', 'Miami, FL 33101'])

In [6]: cities
Out[6]:
0    Boston, MA 02215
1    Miami, FL 33101
dtype: object

In [7]: cities.str.contains(r' [A-Z]{2} ')
Out[7]:
0    True
1    True
dtype: bool

In [8]: cities.str.match(r' [A-Z]{2} ')
Out[8]:
0    False
1    False
dtype: bool
```

We did not specify the index values, so the `Series` uses zero-based indexes by default (snippet [6]). Snippet [7] uses `contains` to show that both `Series` elements contain substrings that match `'[A-Z]{2}'`. Snippet [8] uses `match` to show that neither element's value matches that pattern in its entirety, because each has other characters in its complete value.

Reformatting Your Data

We've discussed data cleaning. Now let's consider munging data into a different format. As a simple example, assume that an application requires U.S. phone numbers in the format `###-###-####`, with hyphens separating each group of digits. The phone numbers have been provided to us as 10-digit strings without hyphens. Let's create the `DataFrame`:

[lick here to view code image](#)

```
In [9]: contacts = [['Mike Green', 'demo1@deitel.com', '5555555555'],
...:                ['Sue Brown', 'demo2@deitel.com', '5555551234']]
...:

In [10]: contactsdf = pd.DataFrame(contacts,
...:                               columns=['Name', 'Email', 'Phone'])
...:

In [11]: contactsdf
Out[11]:
```

	Name	Email	Phone
0	Mike Green	demo1@deitel.com	5555555555
1	Sue Brown	demo2@deitel.com	5555551234

In this `DataFrame`, we specified column indices via the `columns` keyword argument but did *not* specify row indices, so the rows are indexed from 0. Also, the output shows the column values right aligned by default. This differs from Python formatting in which numbers in a field are *right aligned* by default but non-numeric values are *left aligned* by default.

Now, let's munge the data with a little more functional-style programming. We can *map* the phone numbers to the proper format by calling the `Series` method `map` on the `DataFrame`'s `'Phone'` column. Method `map`'s argument is a *function* that receives a value and returns the *mapped* value. The function `get_formatted_phone` maps 10 consecutive digits into the format `###-###-####`:

[lick here to view code image](#)

```

n [12]: import re

In [13]: def get_formatted_phone(value):
...:     result = re.fullmatch(r'(\d{3})(\d{3})(\d{4})', value)
...:     return '-'.join(result.groups()) if result else value
...:
...:

```

The regular expression in the block's first statement matches *only* 10 consecutive digits. It captures substrings containing the first three digits, the next three digits and the last four digits. The `return` statement operates as follows:

- If `result` is `None`, we simply return `value` unmodified.
- Otherwise, we call `result.groups()` to get a tuple containing the captured substrings and pass that tuple to string method `join` to concatenate the elements, separating each from the next with `'-'` to form the mapped phone number.

`Series` method `map` returns a new `Series` containing the results of calling its function argument for each value in the column. Snippet [15] displays the result, including the column's name and type:

[lick here to view code image](#)

```

In [14]: formatted_phone = contactsdf['Phone'].map(get_formatted_phone)

In [15]: formatted_phone
0      555-555-5555
1      555-555-1234
Name: Phone, dtype: object

```

Once you've confirmed that the data is in the correct format, you can update it in the original `DataFrame` by assigning the new `Series` to the `'Phone'` column:

[lick here to view code image](#)

```

In [16]: contactsdf['Phone'] = formatted_phone

In [17]: contactsdf
Out[17]:

```

	Name	Email	Phone
0	Mike Green	demo1@deitel.com	555-555-5555
1	Sue Brown	demo2@deitel.com	555-555-1234

e'll continue our pandas discussion in the next chapter's Intro to Data Science section, and we'll use pandas in several later chapters.

8.14 WRAP-UP

In this chapter, we presented various string formatting and processing capabilities. You formatted data in f-strings and with the string method `format`. We showed the augmented assignments for concatenating and repeating strings. You used string methods to remove whitespace from the beginning and end of strings and to change their case. We discussed additional methods for splitting strings and for joining iterables of strings. We introduced various character-testing methods.

We showed raw strings that treat backslashes (`\`) as literal characters rather than the beginning of escape sequences. These were particularly useful for defining regular expressions, which often contain many backslashes.

Next, we introduced the powerful pattern-matching capabilities of regular expressions with functions from the `re` module. We used the `fullmatch` function to ensure that an entire string matched a pattern, which is useful for validating data. We showed how to use the `replace` function to search for and replace substrings. We used the `split` function to tokenize strings based on delimiters that match a regular expression pattern. Then we showed various ways to search for patterns in strings and to access the resulting matches.

In the Intro to Data Science section, we introduced the synonyms data munging and data wrangling and showed a sample data munging operation, namely transforming data. We continued our discussion of Panda's `Series` and `DataFrames` by using regular expressions to validate and munge data.

In the next chapter, we'll continue using various string-processing capabilities as we introduce reading text from files and writing text to files. We'll use the `csv` module for manipulating comma-separated value (CSV) files. We'll also introduce exception handling so we can process exceptions as they occur, rather than displaying a traceback.

. Files and Exceptions

Objectives

In this chapter, you'll:

- Understand the notions of files and persistent data.
- Read, write and update files.
- Read and write CSV files, a common format for machine-learning datasets.
- Serialize objects into the JSON data-interchange format—commonly used to transmit over the Internet—and deserialize JSON into objects.
- Use the `with` statement to ensure that resources are properly released, avoiding “resource leaks”.
- Use the `try` statement to delimit code in which exceptions may occur and handle those exceptions with associated `except` clauses.
- Use the `try` statement's `else` clause to execute code when no exceptions occur in the `try` suite.
- Use the `try` statement's `finally` clause to execute code regardless of whether an exception occurs in the `try`.
- `raise` exceptions to indicate runtime problems.
- Understand the traceback of functions and methods that led to an exception.
- Use `pandas` to load into a `DataFrame` and process the Titanic Disaster CSV dataset.

Outline

.1 Introduction

.2 Files

.3 Text-File Processing

.3.1 Writing to a Text File: Introducing the `with` Statement

.3.2 Reading Data from a Text File

.4 Updating Text Files

.5 Serialization with JSON

.6 Focus on Security: `pickle` Serialization and Deserialization

.7 Additional Notes Regarding Files

.8 Handling Exceptions

.8.1 Division by Zero and Invalid Input

.8.2 `try` Statements

.8.3 Catching Multiple Exceptions in One `except` Clause

.8.4 What Exceptions Does a Function or Method Raise?

.8.5 What Code Should Be Placed in a `try` Suite?

.9 `finally` Clause

.10 Explicitly Raising an Exception

.11 (Optional) Stack Unwinding and Tracebacks

.12 Intro to Data Science: Working with CSV Files

.12.1 Python Standard Library Module `csv`

.12.2 Reading CSV Files into Pandas `DataFrames`

.12.3 Reading the Titanic Disaster Dataset

9.1 INTRODUCTION

Variables, lists, tuples, dictionaries, sets, arrays, pandas `Series` and pandas `DataFrames` offer only *temporary* data storage. The data is lost when a local variable “goes out of scope” or when the program terminates. **Files** provide long-term retention of typically large amounts of data, even after the program that created the data terminates, so data maintained in files is persistent. Computers store files on secondary storage devices, including solid-state drives, hard disks and more. In this chapter, we explain how Python programs create, update and process data files.

We consider text files in several popular formats—plain text, JSON (JavaScript Object Notation) and CSV (comma-separated values). We’ll use JSON to serialize and deserialize objects to facilitate saving those objects to secondary storage and transmitting them over the Internet. Be sure to read this chapter’s Intro to Data Science section in which we’ll use both the Python Standard Library’s `csv` module and pandas to load and manipulate CSV data. In particular, we’ll look at the CSV version of the Titanic disaster dataset. We’ll use many popular datasets in upcoming data-science case-study chapters on natural language processing, data mining Twitter, IBM Watson, machine learning, deep learning and big data.

As part of our continuing emphasis on Python security, we’ll discuss the security vulnerabilities of serializing and deserializing data with the Python Standard Library’s `pickle` module. We recommend JSON serialization in preference to `pickle`.

We also introduce **exception handling**. An exception indicates an execution-time problem. You’ve seen exceptions of types `ZeroDivisionError`, `NameError`, `ValueError`, `StatisticsError`, `TypeError`, `IndexError`, `KeyError` and `RuntimeError`. We’ll show how to deal with exceptions as they occur by using `try` statements and associated `except` clauses to *handle* exceptions. We’ll also discuss the `try` statement’s `else` and `finally` clauses. The features presented here help you write *robust, fault-tolerant* programs that can deal with problems and continue executing or *terminate gracefully*.

programs typically request and release resources (such as files) during program execution. Often, these are in limited supply or can be used only by one program at a time. We show how to guarantee that after a program uses a resource, it's released for use by other programs, even if an exception has occurred. You'll use the `with` statement for this purpose.

9.2 FILES

Python views a **text file** as a sequence of characters and a **binary file** (for images, videos and more) as a sequence of bytes. As in lists and arrays, the first character in a text file and byte in a binary file is located at position 0, so in a file of n characters or bytes, the highest position number is $n - 1$. The diagram below shows a conceptual view of a file:



For each file you **open**, Python creates a **file object** that you'll use to interact with the file.

End of File

Every operating system provides a mechanism to denote the end of a file. Some represent it with an **end-of-file marker** (as in the preceding figure), while others might maintain a count of the total characters or bytes in the file. Programming languages generally hide these operating-system details from you.

Standard File Objects

When a Python program begins execution, it creates three **standard file objects**:

- `sys.stdin`—the **standard input file object**
- `sys.stdout`—the **standard output file object**, and
- `sys.stderr`—the **standard error file object**.

Though these are considered file objects, they do not read from or write to files by default. The `input` function implicitly uses `sys.stdin` to get user input from the keyboard. Function `print` implicitly outputs to `sys.stdout`, which appears in the command line. Python implicitly outputs program errors and tracebacks to

`sys.stderr`, which also appears in the command line. You must import the `sys` module if you need to refer to these objects explicitly in your code, but this is rare.

9.3 TEXT-FILE PROCESSING

In this section, we'll write a simple text file that might be used by an accounts-receivable system to track the money owed by a company's clients. We'll then read that text file to confirm that it contains the data. For each client, we'll store the client's account number, last name and account balance owed to the company. Together, these data fields represent a client **record**. Python imposes no structure on a file, so notions such as records do not exist natively in Python. Programmers must structure files to meet their applications' requirements. We'll create and maintain this file in order by account number. In this sense, the account number may be thought of as a **record key**. For this chapter, we assume that you launch IPython from the `ch09` examples folder.

9.3.1 Writing to a Text File: Introducing the `with` Statement

Let's create an `accounts.txt` file and write five client records to the file. Generally, records in text files are stored one per line, so we end each record with a newline character:

[lick here to view code image](#)

```
In [1]: with open('accounts.txt', mode='w') as accounts:
...:     accounts.write('100 Jones 24.98\n')
...:     accounts.write('200 Doe 345.67\n')
...:     accounts.write('300 White 0.00\n')
...:     accounts.write('400 Stone -42.16\n')
...:     accounts.write('500 Rich 224.62\n')
...:
```

You can also write to a file with `print` (which automatically outputs a `\n`), as in

[lick here to view code image](#)

```
print('100 Jones 24.98', file=accounts)
```

The `with` Statement

Many applications *acquire* resources, such as files, network connections, database